# 3 Introduction to Spring AOP

## 3.1 Objectives

- To introduce the Aspect Oriented Programming in Spring
- To know about Cross – cutting concerns, Aspects and Advices in AOP
- To understand the use of interfaces to implement the various advices in AOP
- To know about point cuts and the XML based configurations of Aspect Oriented Programming

## 3.2 Audience

This document is designed for Spring Learners and programmers with a need to understand the Aspect Oriented Programming components of the Spring framework in detail. The trainees who have understood the features of the Spring Framework with a good understanding of IoC concepts and Dependency Injection of the Spring Core Containers are the intended audience for this document.

## 3.3 Prerequisites

Before proceeding with this document you should have a fair knowledge about the Spring Architecture, Dependency Injection and XML concepts.

## 3.4 Course Content

### 3.4.1 Spring AOP

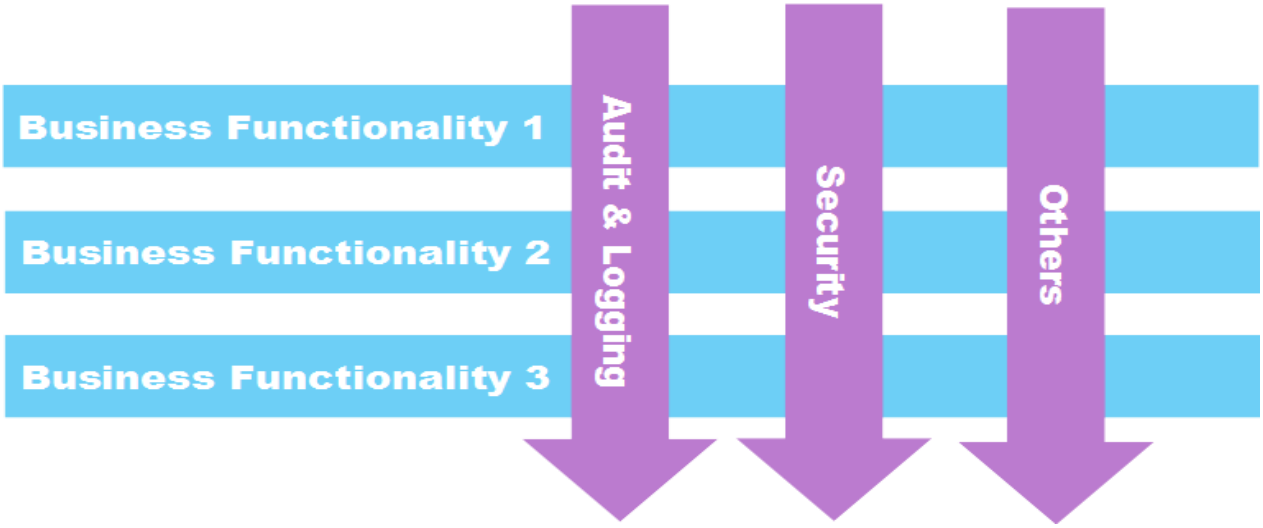
One of the key components of the Spring Framework is the Aspect Oriented programming (AOP) framework. Aspect Oriented Programming entails breaking down program logic into distinct parts called concerns.Aspects enable modularization of concerns like transaction management that cut across multiple functions and objects. The key unit of modularity in OOP is the class,

whereas in AOP the unit of modularity is an aspect. AOP complements Spring IoC to provide a very capable middleware solution.Spring AOP module provides interceptors to intercept an application, for example, when a method is executed, you can add extra functionality before or after the method execution. Note that AOP and OOP are not exclusive. In contrary, they complement each other and their combined use can help us produce robust and maintainable software.

### 3.4.2 Cross-Cutting Concerns

The functions that span multiple points of an application are called cross-cutting concerns and these are conceptually separate from the application's business logic. There are various common good examples of aspects or cross-cutting concerns like logging, auditing, declarative transactions, security, and caching etc.,.

**Figure 1: Cross-Cutting concerns in applications**



### 3.4.3 Aspects

A modularization of a concern that cuts across multiple objects is an Aspect. It is similar to Classes in OOP where the Classes are the unit of modularity. In AOP, Aspects are the unit of modularity. Transaction management is a good example of a crosscutting concern in J2EE applications. In Spring AOP, aspects are implemented using regular classes (the schema-based approach) or regular classes annotated with the @Aspect annotation (@AspectJ style).

**Example:**

```
<bean id="myAspect" class="com.tcs.ilp.spring.examples.MyCustomAspect">
  <!-- configure properties of aspect here as normal -->
</bean>
```

Create the bean class MyCustomAspect as specified and make above bean entry in spring container xml file.

The aspect 'myAspect' will be associated with advices in the <aop:Config> element of the bean configuration file. We will see the same in the forthcoming sections.

### 3.4.4 Advices

This is the actual action to be taken either before or after the method execution. This is actual piece of code that is invoked during program execution by Spring AOP framework. SpringAOP supports several types of advices, namely:

**Before:** This advice weaves the aspect before method call.

**After:** This advice weaves the aspect after the method call is completed even with Exception

**AfterReturning:** This advice weaves the aspect after method call is completed and only if no Exception is thrown during method execution

**AfterThrowing:** This advice weaves the aspect when method throws an Exception.

**Around:** This advice weaves the aspect before and after method call.

There are 2 approaches in which Spring supports aspect oriented programming.

- Declaratively define advices in spring application context XML file

- Programmatically implementing various advices defined by org.springframework.aop

Disadvantage of the second approach is that, AOP client code becomes dependent on Spring framework AOP libraries

### 3.4.5 Joinpoints

A joinpoint is an execution point during the execution of a program, such as the execution of a method or the handling of an exception. In Spring AOP, a joinpoint always represents a method execution. Joinpoint information is available in advice bodies by declaring a parameter of type org.aspectj.lang.JoinPoint. In the example given below, the getCustomerId() method of the Customer class in the package com.tcs.ilp.spring.examples is a joinpoint. So associated advices will run based on this method call. A more generic term for joinpoints are pointcuts.

### 3.4.6 Pointcuts

This is a set of one or more joinpoints where an advice should be executed. Advice is associated with a pointcut expression and runs at any join point matched by the pointcut expression(for example, the execution of a method with a certain name). The concept of join points as matched by pointcut expressions is central to AOP: Spring uses the AspectJ pointcut language by default.

**Try it yourself and verify the output:**

**Example:**

Create MyCustomAspect, Customer classes as below

```
package com.tcs.ilp.spring.examples;

public class MyCustomAspect {
    public MyCustomAspect() {
    }

    public void beforeAdviceMethod(){
        System.out.println("Before method call....got control");
    }
    public void afterAdviceMethod(){
        System.out.println("After method call....again got control");
    }
}
```

```
package com.tcs.ilp.spring.examples;

public class Customer {
    private int customerId;

    public Customer() {
    }

    public int getCustomerId() {
        System.out.println(" Going to give customerID.......");
        return customerId;
    }
}
```

So complete spring context file will now look like below:

```
<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:aop="http://www.springframework.org/schema/aop"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-3.3.xsd
   http://www.springframework.org/schema/aop
         http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="myAspect" class="com.tcs.ilp.spring.examples.MyCustomAspect">
</bean>
<bean id="customer" class="com.tcs.ilp.spring.examples.Customer">
</bean>
<aop:config>
<aop:aspect id="aspectLog" ref="myAspect">
         <aop:pointcut
                  expression="execution(*com.tcs.ilp.spring.examples.Customer.getCustomerId(..))"
                  id="custIdMethodInvocation"/>
         <aop:before pointcut-ref="custIdMethodInvocation" method="beforeAdviceMethod"/>
         <aop:after pointcut-ref="custIdMethodInvocation" method="afterAdviceMethod"/> </aop:aspect>
</aop:config>
```

Note that in the spring context xml, xsi:schemaLocation now refers to aop and spring-aop.xsd

In this example we have the various advices associated to a particular aspect called 'myAspect'. We also notice a <aop:pointCut> element which is referred all along each advice definition.

The output of below client program “SimpleAppln” is as below:

```
package com.tcs.ilp.spring.examples;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;

public class SimpleAppln {
        public static void main(String[] args) {
            ApplicationContext context = new FileSystemXmlApplicationContext
                                    ("D:\\springBeans.xml");

            Customer customer = (Customer ) context.getBean("customer");
            customer.getCustomerId();
        }
  }
```

```
<terminated> SimpleAppln [Java Application] C:\Program Files\Java\jdk1.6.0_29\bin\javaw.exe (May 29, 2014 3:53:18 PM)
May 29, 2014 3:53:22 PM org.springframework.context.support.AbstractApplicationContext prepareRefresh
INFO: Refreshing org.springframework.context.support.FileSystemXmlApplicationContext@32fb4f: startup date [
May 29, 2014 3:53:23 PM org.springframework.beans.factory.xml.XmlBeanDefinitionReader loadBeanDefinitions
INFO: Loading XML bean definitions from file [D:\springBeans.xml]
May 29, 2014 3:53:25 PM org.springframework.beans.factory.support.DefaultListableBeanFactory preInstantiate
INFO: Pre-instantiating singletons in org.springframework.beans.factory.support.DefaultListableBeanFactory@
Before method call....got control
 Going to give customerID.......
After method call....again got control
```

Note that the method names viz; "beforeAdviceMethod", "afterAdviceMethod" are the names of the methods defined in the class com.tcs.ilp.spring.examples.MyCustomAspect as identified by ref="myAspect" in <aop:aspect> tag

Hence these methods will be executed as per the advice defined, whenever any method of Customer class is invoked, as seen in above output

We can still do generalization using the “expression” attribute of the pointcut as below:

**Example:**

```
<aop:config>
        <aop:aspect id="aspectLog" ref="myAspect">
            <aop:pointcut expression="execution(*
com.tcs.ilp.spring.examples.Customer.*.(..))" id="allMethodInvocations"/>
        <aop:before pointcut-ref="allMethodInvocations" method="beforeAdviceMethod"/>
            ...............................
```

This indicates that any advice defined here with pointcut-ref="allMethodInvocations" would get executed on all methods of the Customer class. The joinpoints, therefore, are the methods in the class and the group of these joinpoints is the pointcut with the identifier 'allMethodInvocations'.

### 3.4.7 Advices using Interface Implementations - Examples

#### 3.4.7.1 Before Advice – An Example

**A functional Interface and its implementation**

```
public interface Emart {
        Book  buyBooks (Customer customer) throws EMartException;
}
public class XYZEMart implements Emart {
        public Book  buyBooks (Customer customer) throws EMartException  {
            return new Book();}
}
```

**AOP Interface and its implementation**

```
public interface MethodBeforeAdvice {
```

```
        void before(Method method, Object[] args, Object target) throws Throwable
}
```

**An application to make use of above interfaces/implementations:**

```
import org.springframework.aop.MethodBeforeAdvice;
public class WelcomeAdvice implements MethodBeforeAdvice {
    public void before(Method method, Object[] args, Object target) {
        Customer customer = (Customer) args[0];
        System.out.println("Hello " + customer.getName());
    }
}
```

**Configuration for the application with 'WelcomeAdvice ' applied:**

```
<beans>
    <bean id="EMartTarget" class="XYZEMart"/>
    <bean id="welcomeAdvice" class="WelcomeAdvice"/>
    <bean id="EMart" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="proxyInterfaces">
            <value>EMart</value>
        </property>
        <property name="interceptorNames">
            <list>
                <value>welcomeAdvice</value>
            </list>
        </property>
        <property name="target">
```

```
                <ref bean="EMartTarget"/>
            </property>
        </bean>
</beans>
```

### 3.4.7.2 After Returning Advice – An Example

**A functional Interface and its implementation:**

```
public interface Emart {
        Book  buyBooks (Customer customer) throws EMartException;
}
public class XYZEMart implements Emart {
        public Book  buyBooks (Customer customer) throws EMartException  {
                return new Book();}
}
```

**AOP Interface and its implementation:**

```
public interface MethodAfterReturningAdvice {
                void afterReturning(Method method, Object[] args, Object target) throws Throwable
}
```

**An application to make use of above interfaces/implementations:**

```
import org.springframework.aop.MethodAfterReturningAdvice;
public class WelcomeAdvice implements MethodAfterReturningAdvice{
        public void afterReturning(Method method, Object[] args, Object target) {
                Customer customer = (Customer) args[0];
```

```
        System.out.println("Hello " + customer.getName());
    }
}
```

**Configuration for the application with 'WelcomeAdvice ' applied:**

The Same configuration XML as above example.

### 3.4.7.3 Around Advice – An Example

**A functional Interface and its implementation:**

```
public interface MethodInterceptor extends Interceptor {
    Object invoke(MethodInvocation invocation) throws Throwable;
}
```

**AOP Interface and its implementation:**

```
public class DebugInterceptor implements MethodInterceptor {
    public Object invoke(MethodInvocation invocation) throws Throwable {
        System.out.println("Before: invocation=[" + invocation + "]");
        Object rval = invocation.proceed();
        System.out.println("Invocation returned");
        return rval;
    }
}
```

### 3.4.7.4 Throws Advice – An Example :

**AOP Interface and its implementation:**

```
public class ServletThrowsAdviceWithArguments implements ThrowsAdvice {
    public void afterThrowing(Method m, Object[] args, Object target, ServletException ex)
    {       // Do something with all arguments
```

```
            System.out.println("Exception Thrown : " + args[0]);
        }
}
```

### 3.4.8 Other AOP Frameworks

There are many AOP frameworks that have come up as extensions to standard Spring AOP. Eg AspectJ, AspectWerkz, JbossAOP, EAOP, DynamicAspects, etc.

AspectJ is an aspect-oriented programming (AOP) extension created at PARC for the Java programming language. AspectJ refers to a style of declaring aspects as regular Java classes annotated with Java 5 annotations. The AspectJ style was introduced by the AspectJ Projectas part of the AspectJ 5 release. Spring 2.0 interprets the same annotations as AspectJ 5, using a library supplied by AspectJ for pointcut parsing and matching. The AOP runtime is still pure Spring AOP though, and there is no dependency on the AspectJ compiler or weaver.